Process Synchronization

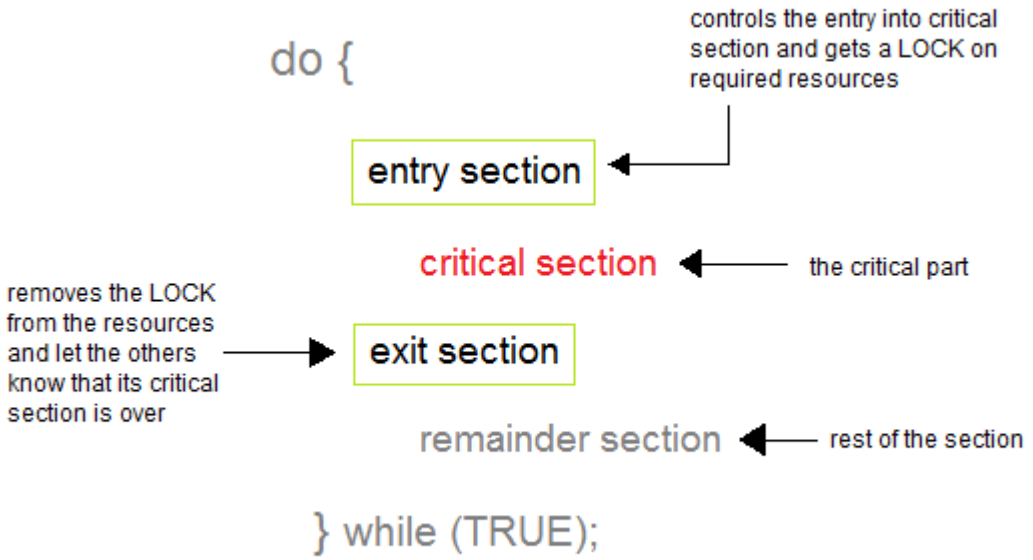
Dr Nishant Jain, MITS-DU Gwalior

- Process Synchronization means coordinating the execution of processes such that no two processes access the same shared resources and data.
- It is required in a multi-process system where multiple processes run together, and more than one process tries to gain access to the same shared resource or data at the same time.

For example: A process P1 tries changing data in a particular memory location. At the same time another process P2 tries reading data from the same memory location. Thus, there is a high probability that the data being read by the second process is incorrect.

Sections of a Program

1. **Entry Section:** This decides the entry of any process.
2. **Critical Section:** This allows a process to enter and modify the shared variable.
3. **Exit Section:** This allows the process waiting in the Entry Section, to enter the Critical Sections and makes sure that the process is removed through this section once it's done executing.
4. **Remainder Section:** Parts of the Code, not present in the above three sections are collectively called Remainder Section.



Critical Section Problem

- A Critical Section is a code segment that accesses shared variables and must be executed as an atomic action.
- It means that in a group of cooperating processes, at a given point of time, only one process must be executing its critical section.
- If any other process also wants to execute its critical section, it must wait until the first one finishes.
- The entry to the critical section is mainly handled by wait() function while the exit from the critical section is controlled by the signal() function.

Race Condition

- When more than one processes execute the same code or access the same memory/shared variable, it is possible that the output or value of the shared variable is inconsistent.
- In this condition, all processes race ahead in order to prove that their output is correct. This situation is known as race condition.
- When multiple processes access and manipulate the same data concurrently the outcome depends on the order in which these processes accessed the shared data. When the output of multiple thread execution differs according to the order in which the threads execute, a race condition occurs.
- We can avoid it if we treat the critical section as an atomic instruction and maintain proper thread synchronization using locks or atomic variables.

Rules for Critical Section

1. **Mutual Exclusion:** Only one process at a time can be executing in their critical section.

2. Progress: If no process is currently executing in their critical section, and one or more processes want to execute their critical section, then only the processes not in their remainder sections can participate in the decision, and the decision cannot be postponed indefinitely. (i.e. processes cannot be blocked forever waiting to get into their critical sections.)

3. Bound Waiting: There exists a limit as to how many other processes can get into their critical sections after a process requests entry into their critical section and before that request is granted. (i.e. a process requesting entry into their critical section will get a turn eventually, and there is a limit as to how many other processes get to go first.)

Solution to The Critical Section

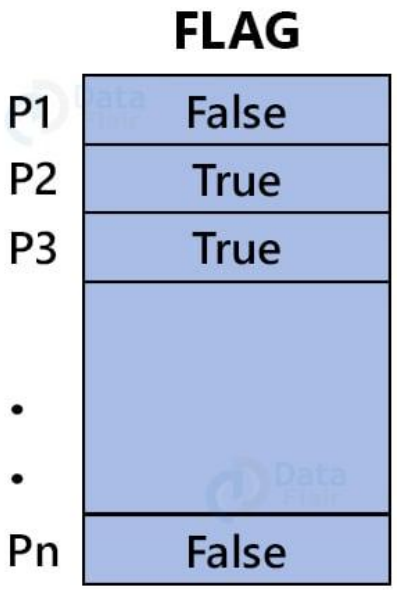
1. Peterson Solution: If a process executes in a critical state, the other process can only execute the rest of the code and vise-versa. This method developed by Peterson is widely used and helps make sure that only one process runs at a specific time in the critical section.

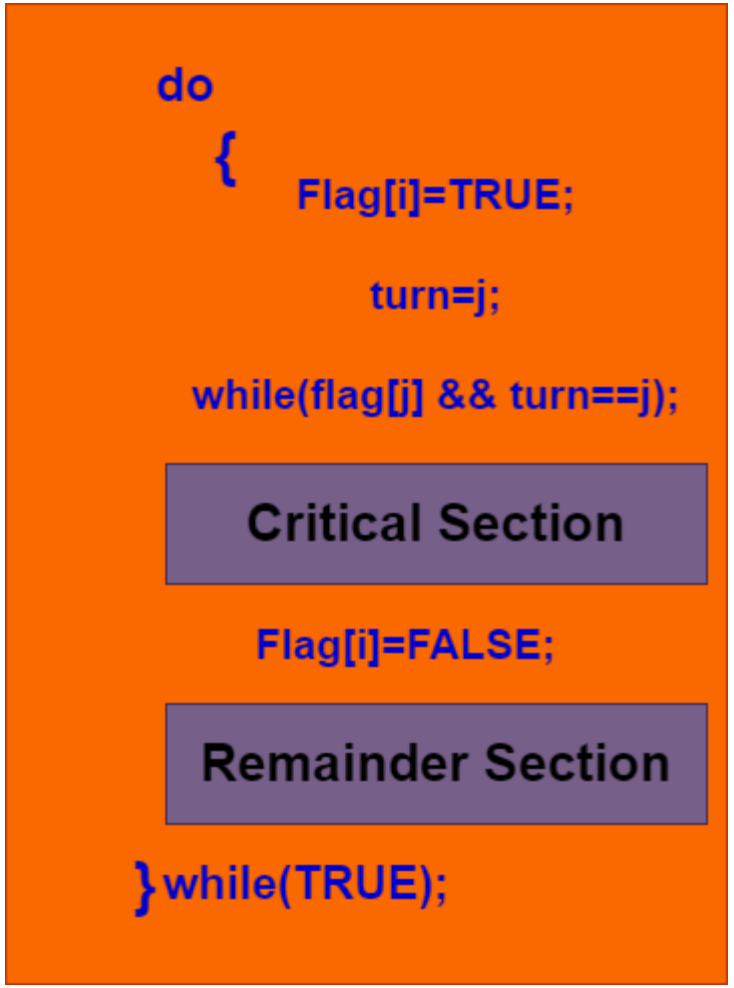
Example: Let there be N processes (P1, P2, … PN) such that at some point of time every process needs to enter the Critical Section. There is a FLAG[] array of size N that is false by default. Therefore, the flag of a process needs to be set true, whenever it wants to enter the critical section.

A variable TURN tells the process number waiting to enter the Critical Section. When a process enters the critical section it changes the TURN to another number from the list of ready processes when it is exiting.

Peterson Solution

```
PROCESS Pi
FLAG[i] = true
while( (turn != i) AND (CS is !free) ){ wait;
}
CRITICAL SECTION FLAG[i] = false
turn = j; //choose another process
```





Solution to The Critical Section

2. Synchronization Hardware: Hardware can also help resolve the problems of critical sections sometimes. Some OS offer lock functionality. This gives a process a lock when it enters the critical section and releases the lock after it leaves a critical section. Due to this other processes can't enter a critical section when a process is already inside.

Solution to The Critical Section

3. Semaphore: Semaphore is simply a variable that is non-negative and shared between threads. It is another algorithm or solution to the critical section problem. It is a signalling mechanism and a thread that is waiting on a semaphore, which can be signalled by another thread.

It uses two atomic operations, 1)wait, and 2) signal for the process synchronization.

Example

```
WAIT ( S ):
while ( S <= 0 );
S = S - 1;
SIGNAL ( S ):
S = S + 1;
```

Types of Semaphore

semaphores can take on one of two forms:

Binary semaphores can take on one of two values, 0 or 1. They can be used to solve the critical section, and can be used as mutexes on systems that do not provide a separate mutex mechanism.

```
do {
   waiting(mutex);

      // critical section

   signal(mutex);

      // remainder section
}while (TRUE);
```

Types of Semaphore

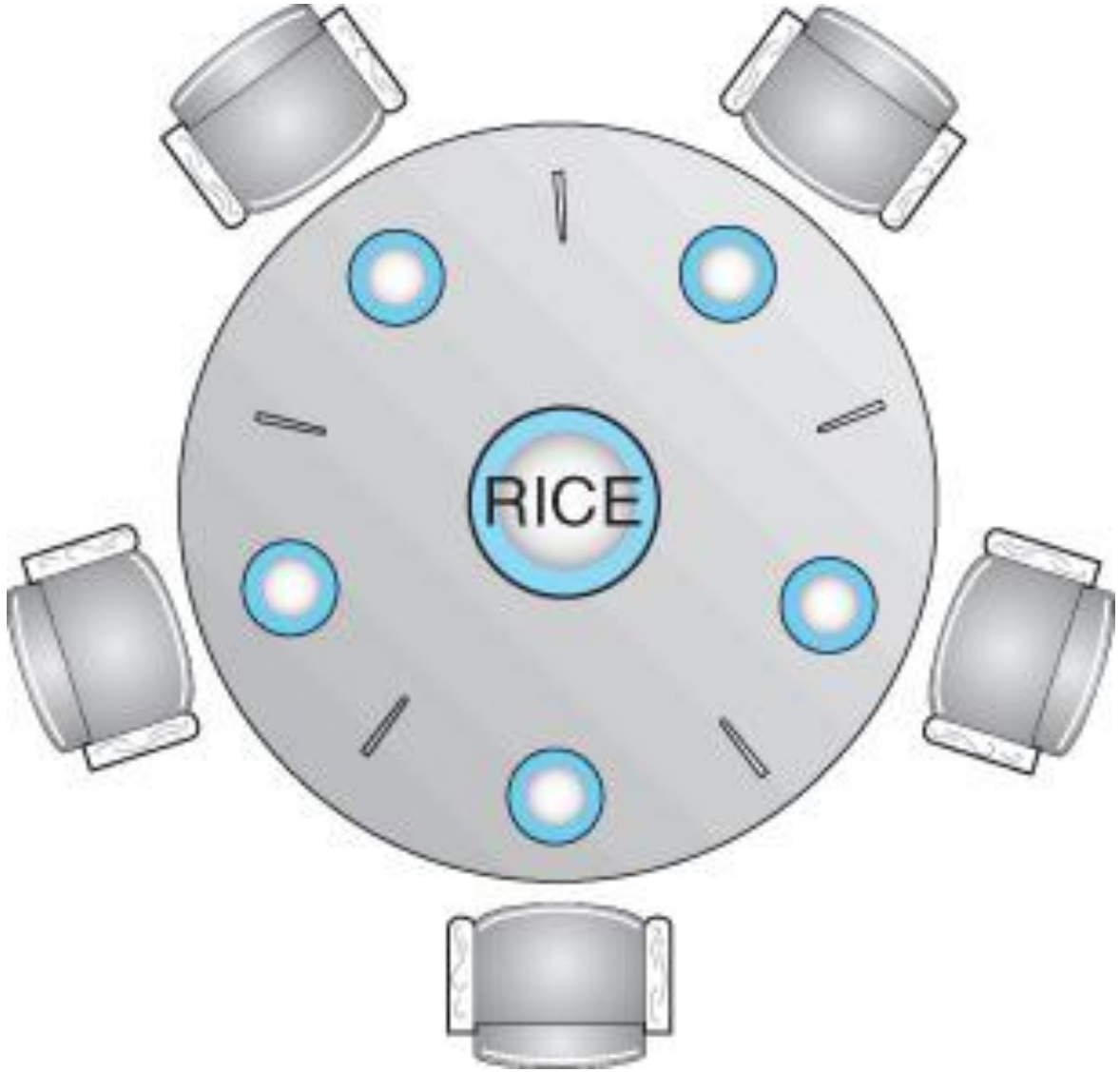
Counting semaphores can take on any integer value, and are usually used to count the number remaining of some limited resource.

- The counter is initialized to the number of such resources available in the system, and whenever the counting semaphore is greater than zero, then a process can enter a critical section and use one of the resources.
- When the counter gets to zero (or negative in some implementations), then the process blocks until another process frees up a resource and increments the counting semaphore with a signal call.

Classic Problems of Synchronization

The Dining-Philosophers Problem

The dining philosophers problem is a classic synchronization problem involving the allocation of limited resources amongst a group of processes in a deadlock-free and starvation-free manner:



Solution to Avoid Deadlock

- The maximum number of philosophers on the table should not exceed four.
- Only when both Forks (left and right) are available at the same moment can a philosopher be permitted to choose their Forks.
- A philosopher in an even position should choose the right Fork first, followed by the left Fork, whereas a philosopher in an odd position should choose the left Fork first, followed by the right Fork.
- The four beginning philosophers (P0, P1, P2, and P3) should choose the left Fork first, then the right Fork, while the philosopher (P4) should choose the right Fork first, then the left fork.

The Readers Writers Problem

- The **readers-writers problem** is related to an object(such as a file or a database) that is shared by numerous processes.
- Some of these processes are readers, meaning they only want to read data from the object, while others are writers, meaning they want to write data into the object.
- The **readers-writers problem** is used to maintain synchronization so that the object data is not corrupted.

The Readers Writers Problem

There are several variations to the readers-writers problem, most centered around relative priorities of readers versus writers.

- The *first* readers-writers problem gives priority to readers. In this problem, if a reader wants access to the data, and there is not already a writer accessing it, then access is granted to the reader. A solution to this problem can lead to starvation of the writers, as there could always be more readers coming along to access the data.
- The *second* readers-writers problem gives priority to the writers. In this problem, when a writer wants access to the data it jumps to the head of the queue - All waiting readers are blocked, and the writer gets access to the data as soon as it becomes available. In this solution the readers may be starved by a steady stream of writers.

The Reader Process

1. The reader requests entry to the critical section.

2. If permitted,

- it increments the number of readers within the critical section. If this reader is the first to enter, the wrt semaphore is locked, preventing writers from entering if any other reader is present.
- it then signals mutex, indicating that any new reader may enter while others are currently reading.
- it leaves the critical section after reading. When departing, it checks to see whether there are any more readers within and if there are, it signals the semaphore "wrt," indicating that the writer can now enter the critical region.

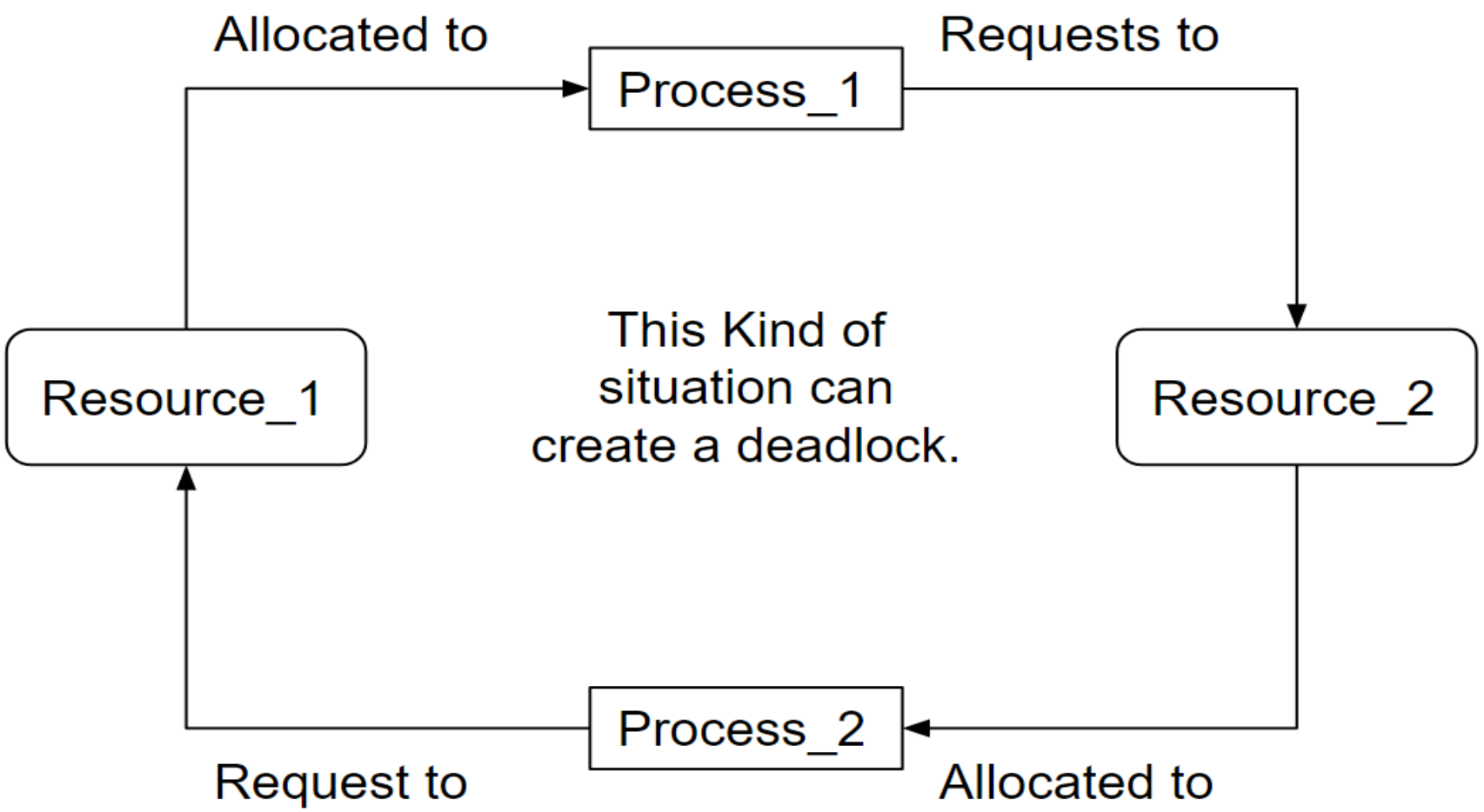
3. If it is not permitted, it will continue to wait.

The Writer Process

1. The writer requests entry to the critical section.
2. If permitted, i.e., wait() returns a true value, it enters and performs the write.
3. If it is not allowed, it will continue to wait.
4. It gets out of the critical section.

Deadlock

- A **deadlock** in an operating system is a condition where two or more processes are waiting for some event, but the event does not occur; in this case, those two processes are stuck in a deadlock state.
- A set of process is in a deadlock state if each process in the set is waiting for an event that can be caused by only another process in the set.
- In other words, each member of the set of deadlock processes is waiting for a resource that can be released only by a deadlock process.


Allocated to
Process_1
Requests to
Resource_1
This Kind of
situation can
create a deadlock.
Resource_2
Process_2
Request to
Allocated to

Resource Utilization

Resources are utilized in the following ways:

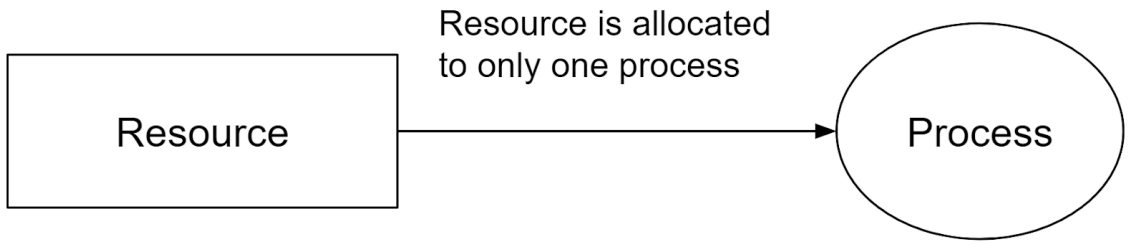
- **Request**: If the requested resources are unavailable, the process must wait until they become available.
- **Use**: During the process, resources are used.
- **Release**: By releasing the resource, the process relinquishes control. As a result, other processes can use it.

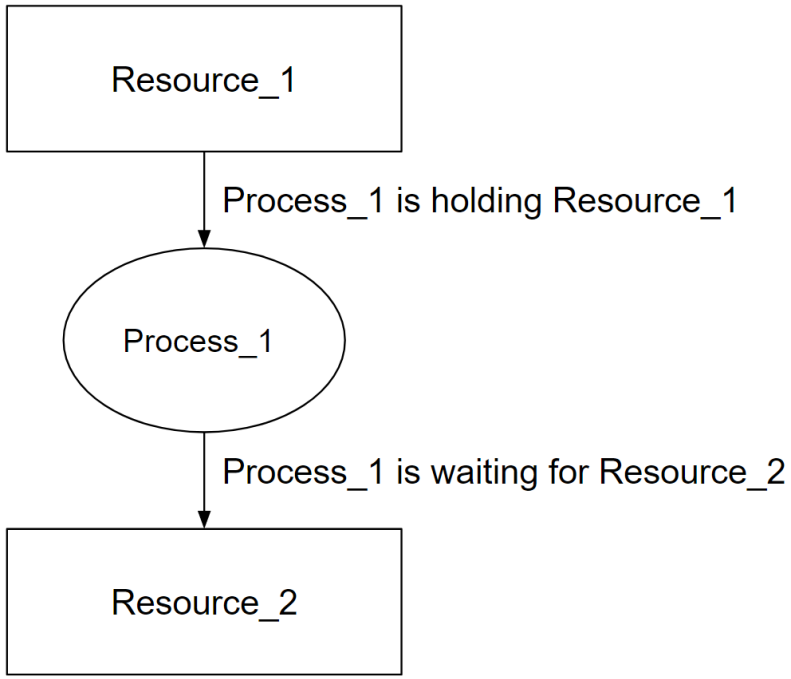
Necessary Conditions for Deadlock

Coffman (1971) identified four conditions that must hold simultaneously for there to be a deadlock.

1. Mutual Exclusion Condition: Any resources you are using should be used mutually exclusively, meaning both processes cannot use the same resource simultaneously. The resources involved are non-shareable.

Explanation: At least one resource must be held in a non-shareable mode, that is, only one process at a time claims exclusive control of the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.





Necessary Conditions for Deadlock

2. Hold and Wait Condition: Requesting process hold already the resources while waiting for requested resources.

Explanation: There must exist a process that is holding a resource already allocated to it while waiting for additional resource that are currently being held by other processes. When this condition exists, the process must be stopped if it simultaneously holds one or more resources while waiting for another resource.

Necessary Conditions for Deadlock



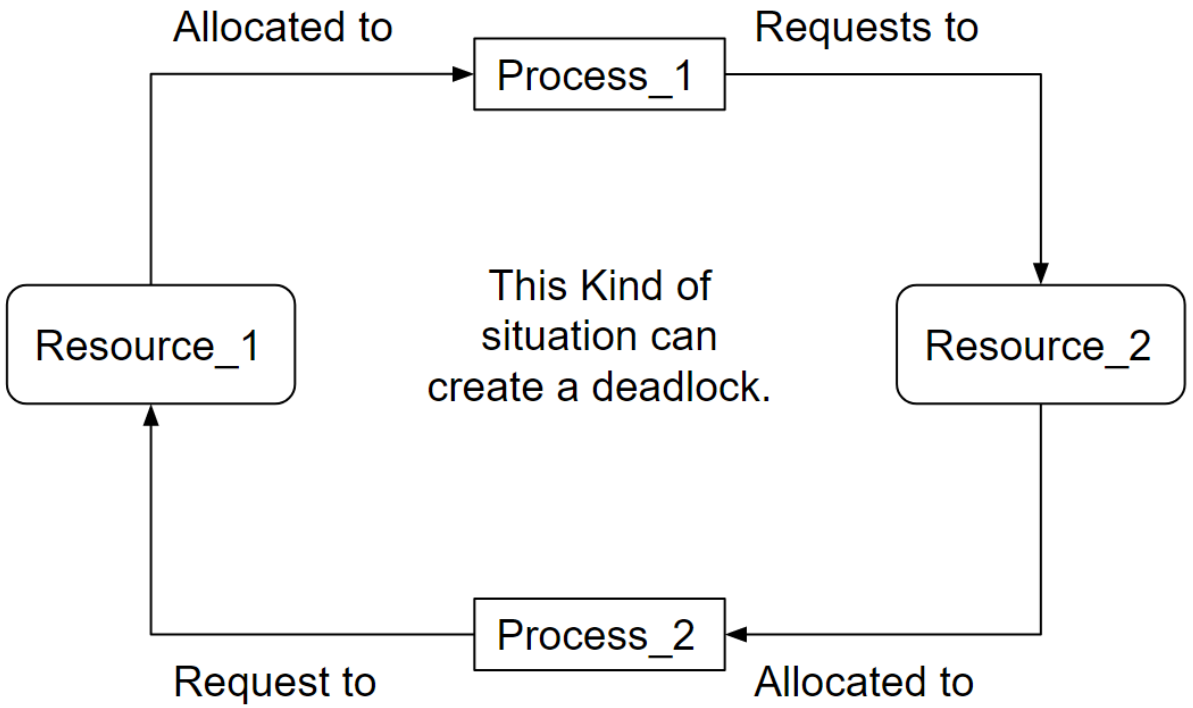
3. **No-Pre-emptive Condition:** Resources already allocated to a process cannot be pre-empted.



Explanation: Resources cannot be removed from the processes are used to completion or released voluntarily by the process holding it.

Necessary Conditions for Deadlock

4. Circular Wait Condition: The processes in the system form a circular list or chain where each process in the list is waiting for a resource held by the next process in the list. There exists a set {P0, P1, …, P0} of waiting processes such that P0 is waiting for a resource that is held by P1, P1 is waiting for a resource that is held by P2, …, Pn–1 is waiting for a resource that is held by Pn, and P0 is waiting for a resource that is held by P0.



Note:

- It is not possible to have a deadlock involving only one single process. The deadlock involves a circular “hold-and-wait” condition between two or more processes, so “one” process cannot hold a resource, yet be waiting for another resource that it is holding.
- In addition, deadlock is not possible between two threads in a process, because it is the process that holds resources, not the thread that is, each thread has access to the resources held by the process.

Important Concept

Consider there are n processes in the system P_1, P_2, P_3, ... , P_n where-

- Process P_1 requires x_1 units of resource R
- Process P_2 requires x_2 units of resource R
- Process P_3 requires x_3 units of resource R and so on.

In worst case,

So,

- Process P_1 holds $x_1 - 1$ units of resource R
- Process P_2 holds $x_2 - 1$ units of resource R
- Process P_3 holds $x_3 - 1$ units of resource R and so on.

Now,

- Had there been one more unit of resource R in the system, system could be ensured deadlock free.
- This is because that unit would be allocated to one of the processes and it would get execute and then release its units.

The number of units that each process holds = One less than its maximum demand

Maximum Number Of Units That Ensures Deadlock-

Maximum number of units of resource R that ensures deadlock

$= (x_1-1) + (x_2-1) + (x_3-1) + \ldots. + (x_n-1)$

$= (x_1 + x_2 + x_3 + \ldots. + x_n) - n$

$= \sum x_i - n$

= Sum of max needs of all n processes – n

Minimum Number Of Units That Ensures No Deadlock-

Minimum number of units of resource R that ensures no deadlock

= One more than maximum number of units of resource R that ensures deadlock

$= (\sum x_i - n) + 1$

Problem-01:

A system is having 3 user processes each requiring 2 units of resource R. The minimum number of units of R such that no deadlock will occur-

1. 3
2. 5
3. **4**
4. 6

Problem-02:

A system is having 10 user processes each requiring 3 units of resource R. The minimum number of units of R such that no deadlock will occur _____?

Problem-03:

If there are 6 units of resource R in the system and each process in the system requires 2 units of resource R, then how many processes can be present at maximum so that no deadlock will occur?

METHODS OF HANDLING DEADLOCK

In general, there are four strategies of dealing with deadlock problem:

1. **Deadlock Prevention**: Prevent deadlock by resource scheduling so as to negate at least one of the four conditions.
2. **Deadlock Avoidance**: Avoid deadlock by careful resource scheduling.
3. **Deadlock Detection and Recovery**: Detect deadlock and when it occurs, take steps to recover.
4. **The Ostrich Approach:** Just ignore the deadlock problem altogether.

DEADLOCK PREVENTION

A deadlock may be prevented by denying any one of the conditions.

Elimination of "Mutual Exclusion" Condition: The mutual exclusion condition must hold for non-sharable resources. That is, several processes cannot simultaneously share a single resource. This condition is difficult to eliminate because some resources, such as the Hard disc drive and printer, are inherently non-shareable. Note that shareable resources like read-only-file do not require mutually exclusive access and thus cannot be involved in deadlock.

Elimination of "Hold and Wait" Condition: There are two possibilities for elimination of the second condition.

- The first alternative is that a process request be granted all of the resources it needs at once, prior to execution.
- The second alternative is to disallow a process from requesting resources whenever it has previously allocated resources.

This strategy requires that all of the resources a process will need must be requested at once. The system must grant resources on "all or none" basis. If the complete set of resources needed by a process is not currently available, then the process must wait until the complete set is available. While the process waits, however, it may not hold any resources. Thus the "wait for" condition is denied and deadlocks cannot occur. This strategy can lead to serious waste of resources.

Elimination of "No-preemption" Condition: The non-preemption condition can be alleviated by forcing a process waiting for a resource that cannot immediately be allocated to relinquish all of its currently held resources, so that other processes may use them to finish.

This strategy requires that when a process that is holding some resources is denied a request for additional resources. The process must release its held resources and, if necessary, request them again together with additional resources. Implementation of this strategy denies the "no-preemptive" condition effectively.

Elimination of "Circular Wait" Condition: The last condition, the circular wait, can be denied by imposing a total ordering on all of the resource types and then forcing, all processes to request the resources in order (increasing or decreasing). This strategy impose a total ordering of all resources types, and to require that each process requests resources in a numerical order (increasing or decreasing) of enumeration. With this rule, the resource allocation graph can never have a cycle.

For example, provide a global numbering of all the resources, as shown

1 ≡ Card reader

2 ≡ Printer

3 ≡ Optical driver

4 ≡ HDD

5 ≡ Card punch

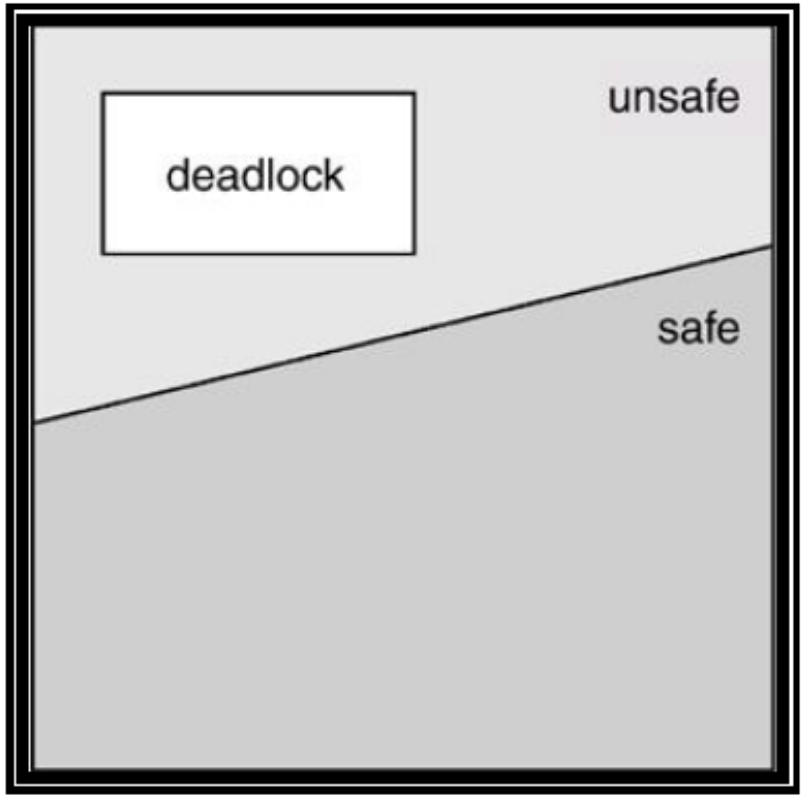
DEADLOCK AVOIDANCE

- This approach to the deadlock problem anticipates deadlock before it actually occurs.
- This approach employs an algorithm to access the possibility that deadlock could occur and acting accordingly.
- If the necessary conditions for a deadlock are in place, it is still possible to avoid deadlock by being careful when resources are allocated.
- It employs the most famous deadlock avoidance algorithm that is the Banker's algorithm.

A deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular wait condition can never exist. The resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes.

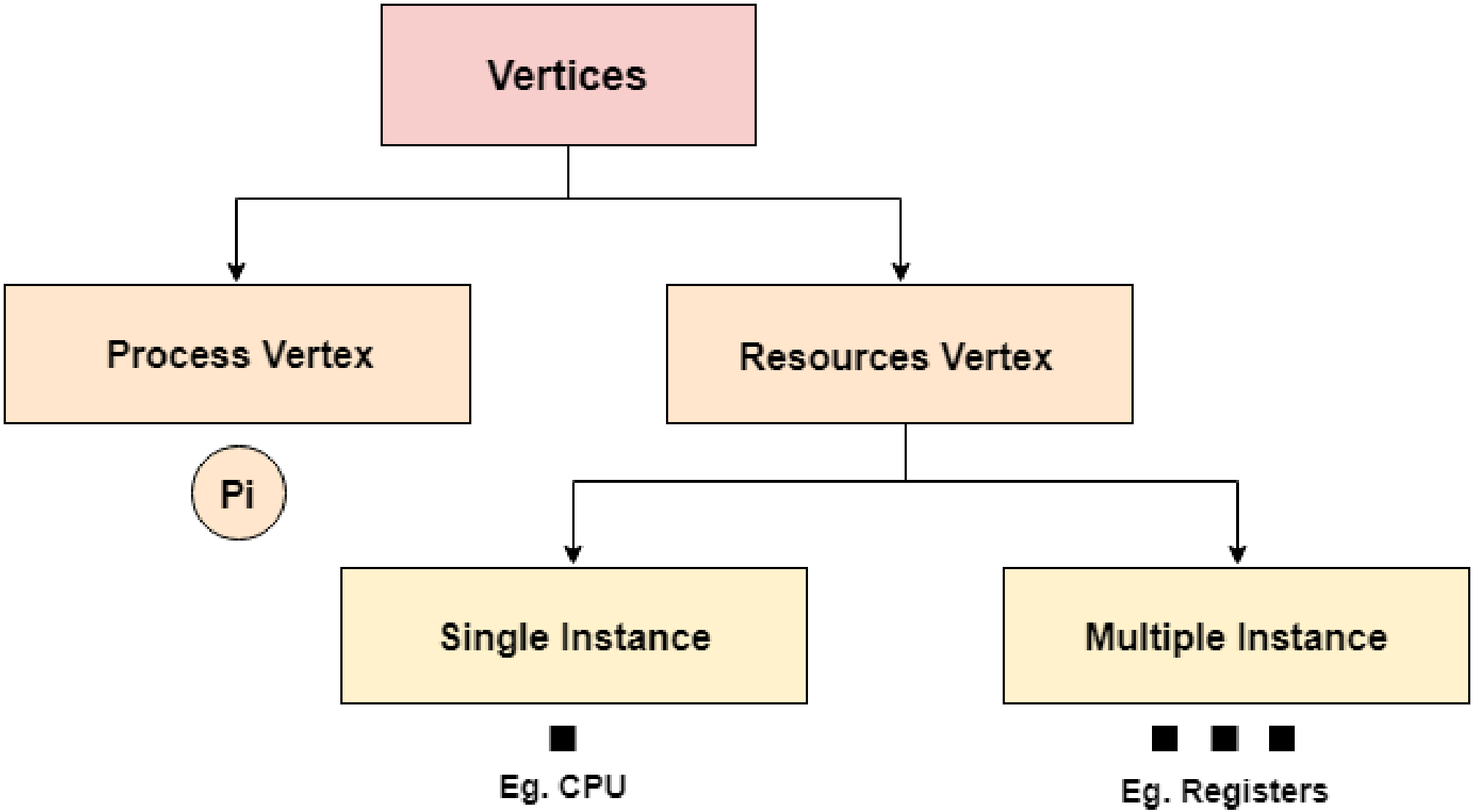
Safe and Unsafe States

- A system is said to be in a **Safe State**, if there is a safe execution sequence. An execution sequence is an ordering for process execution such that each process runs until it terminates or blocked and all request for resources are immediately granted if the resource is available.
- A system is said to be in an **Unsafe State**, if there is no safe execution sequence. An unsafe state may not be deadlocked, but there is at least one sequence of requests from processes that would make the system deadlocked.

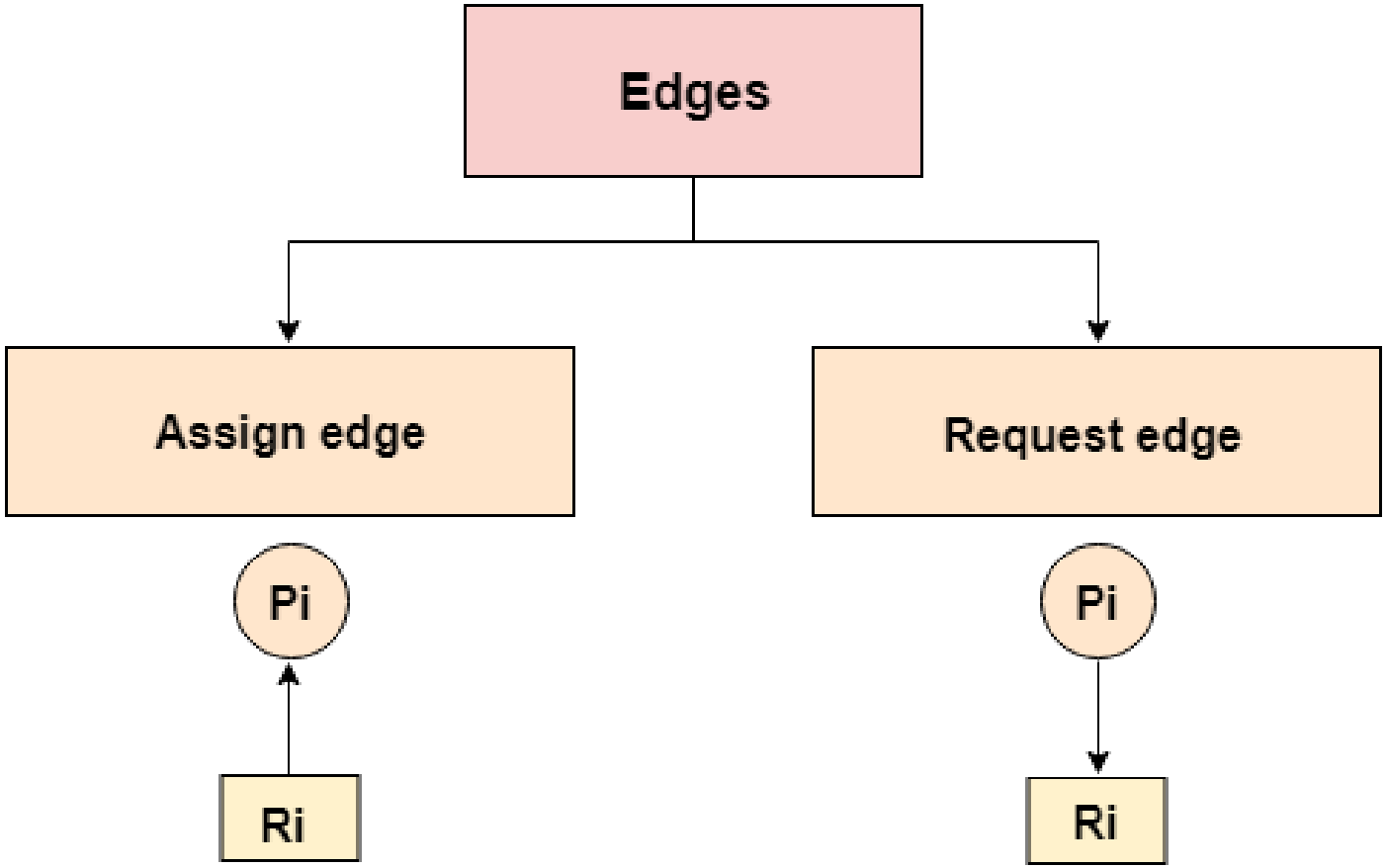


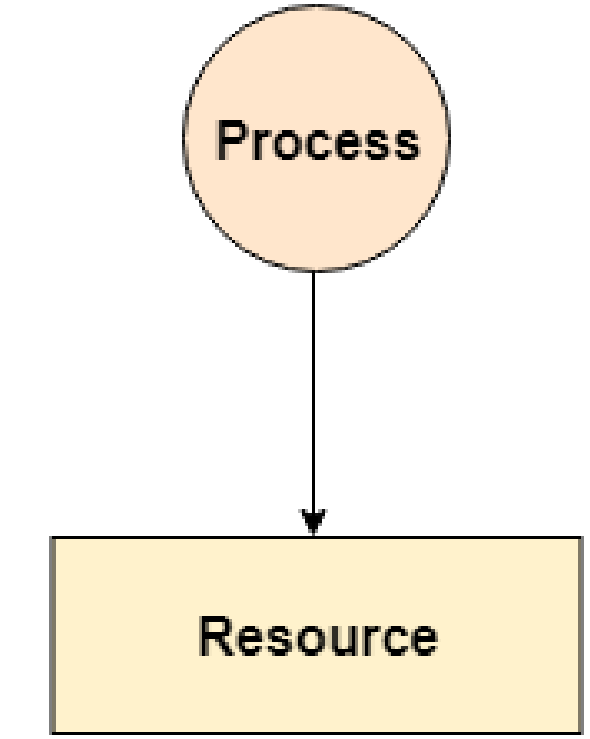
Resource-Allocation Graph

- The resource allocation graph is the pictorial representation of the state of a system.
- The resource allocation graph is the complete information about all the processes which are holding some resources or waiting for some resources.
- It also contains the information about all the instances of all the resources whether they are available or being used by the processes.
- In Resource allocation graph, the process is represented by a Circle while the Resource is represented by a rectangle.

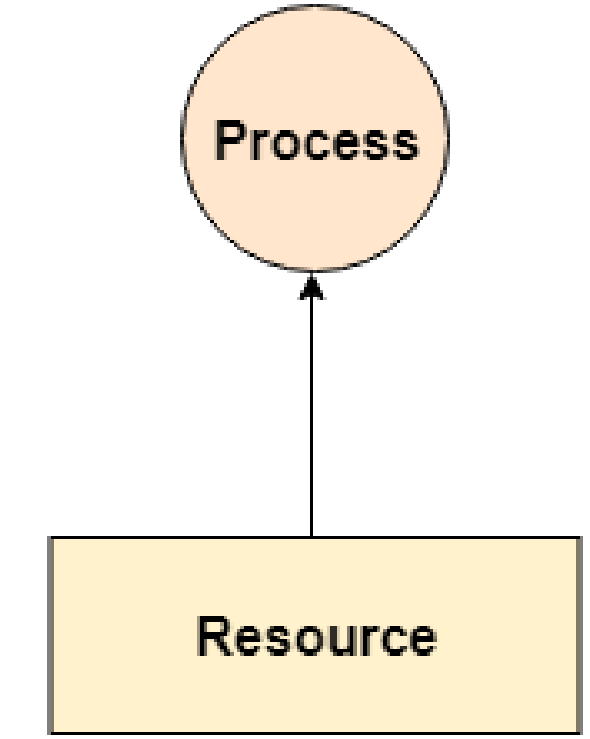

Vertices
Process Vertex
Pi
Resources Vertex
Single Instance
Eg. CPU
Multiple Instance
Eg. Registers

Edges in RAG are also of two types, one represents assignment and other represents the wait of a process for a resource.





Process is requesting for a resource

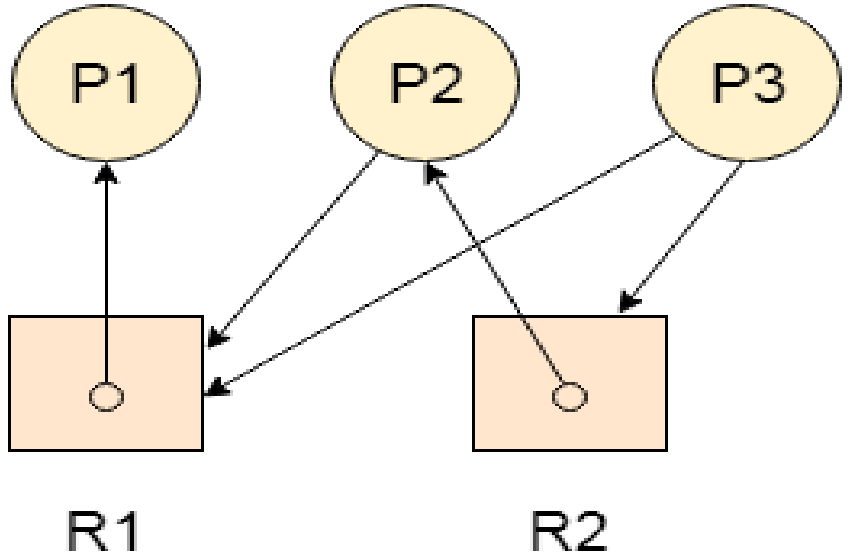


Resource is assigned to process

Example

Let's consider 3 processes P1, P2 and P3, and two types of resources R1 and R2. The resources are having 1 instance each.

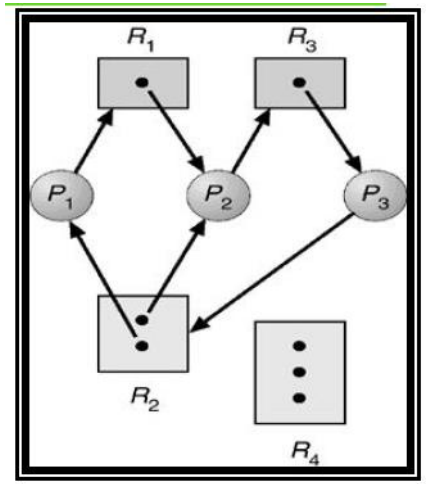
According to the graph, R1 is being used by P1, P2 is holding R2 and waiting for R1, P3 is waiting for R1 as well as R2.

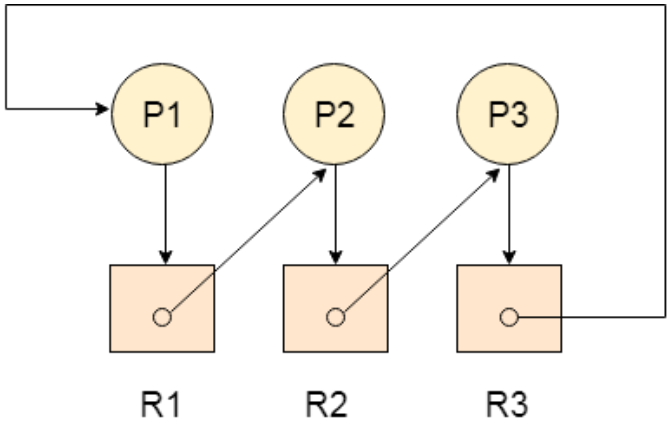


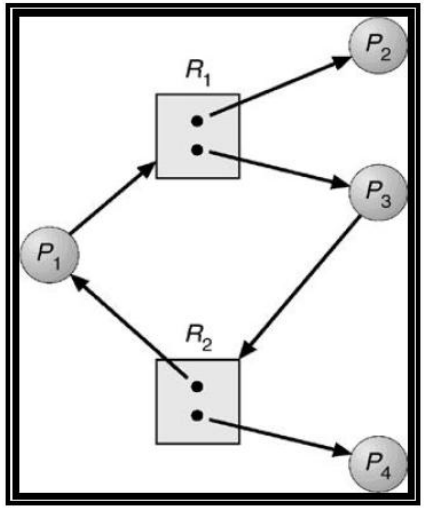
The graph is deadlock free since no cycle is being formed in the graph.

Deadlock Detection using RAG

- If a cycle is being formed in a Resource allocation graph where all the resources have the single instance then the system is deadlocked.
- In Case of Resource allocation graph with multi-instanced resource types, Cycle is a necessary condition of deadlock but not the sufficient condition.







Allocation Matrix

Process	R1	R2	R3
P1	0	0	1
P2	1	0	0
P3	0	1	0

Request Matrix

Process	R1	R2	R3
P1	1	0	0
P2	0	1	0
P3	0	0	1

Banker's algorithm

- The **Banker's algorithm** is a resource allocation & deadlock avoidance algorithm developed by Edsger Dijkstra that test for safety by simulating the allocation of pre-determined maximum possible amounts of all resources.
- It makes a "safe-state" check to test for possible deadlock conditions for all other pending activities, before deciding whether allocation should be allowed to continue.
- The Banker's algorithm is run by the operating system whenever a process requests resources.
- The algorithm prevents deadlock by denying or postponing the request if it determines that accepting the request could put the system in an unsafe state (one where deadlock could occur).

For the Banker's algorithm to work, it needs to know three things:

- How much of each resource could possibly request by each process.
- How much of each resource is currently holding by each process.
- How much of each resource the system currently has available.

Resources may be allocated to a process only if it satisfies the following conditions:

1. request ≤ max, else set error as process has crossed maximum claim made by it.
2. request ≤ available, else process waits until resources are available.

Several data structures must be maintained to implement the banker's algorithm. These data structures encode the state of the resource-allocation system. Let n be the number of processes in the system and m be the number of resource types.

Available: A vector of length m indicates the number of available resources of each type. If Available[j] = k, there are k instances of resource type Rj available.

Max: An n x m matrix defines the maximum demand of each process. If *Max[i,j]* = k, then process *Pi* may request at most k instances of resource type *Rj*.

Allocation: *An n x m* matrix defines the number of resources of each type currently allocated to each process. If *Allocation[i,j]* = k, then process Pi is currently allocated k instances of resource type *Rj*.

Need: An n x m matrix indicates the remaining resource need of each process. If *Need[i,j]* = k, then process Pi may need k more instances of resource type *Ri* to complete its task. Note that *Need[i,j]* = *Max[i,j]* - *Allocafion[i,j]*.

Example:

Let us consider the following snapshot for understanding the banker's algorithm:

1. calculate the content of the need matrix?
2. Check if the system is in a safe state?
3. Determine the total sum of each type of resource?

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

Process	Max	Allocation	Available
	A, B, C, D	A, B, C, D	A, B, C, D
P0	6 0 1 2	4 0 0 1	3 2 1 1
P1	2 7 5 0	1 1 0 0	
P2	2 3 5 6	1 2 5 4	
P3	1 6 5 3	0 6 3 3	
P4	1 6 5 6	0 2 1 2	

Using Banker's algorithm, answer the following questions:-
i) How many resources of type A, B, C, D are there?
ii) What are the contents of need matrix?
iii) Find if the system is in safe state? If it is, find the safe sequence.

Banker's Algorithm consists of two algorithms

1.Safety Algorithm - Safety algorithm check for the safe state of the system. If system is in safe state with any of the resource allocation permutation then deadlock can be avoided.

Steps for Safety Algorithms

1. Let *Work* and *Finish* be vectors of length *m* and *n,* respectively. Initialize

   Work := *Available* and *Finisk[i] :=false* for $i = 1,2, \ldots, n$.

2. Find an *i* such that both

   a. *Finisk[i] =false*

   b. $Need_i < Work$.

If no such ***i*** exists, go to step 4.

3. *Work* := *Work* + $Allocation_i$

   Finisk[i] := *true*

   go to step 2.

4. If *Finish[i]* = *true* for all *i,* then the system is in a safe state.

This algorithm may require an order of $m \times n_2$ operations to decide whether a state is safe.

2. Resource request algorithm - Resource request algorithm checks if the request can be safely granted or not to the process requesting for resources.

If a process Pi request for c instances for resource of type j then it can be represented as Request[i,j]=c.

Steps for Resource request Algorithms

Let $Request_i$ be the request vector for process *Pi.* If $Request_i[j] = k$, then process *Pi* wants *k* instances of resource type *Rj.* When a request for resources is made by process *Pi,* the following actions are taken:

1. If $Request_i < Need_i$, go to step 2. Otherwise, raise an error condition, since the process has exceeded its maximum claim.
2. If $Request_i < Available$, go to step 3. Otherwise, *Pi* must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to process *Pi* by modifying the state as follows:

Available := Available - $Request_i$;

$Allocation_i$:= $Allocation_i$ + $Request_i$;

$Need_i$:= $Need_i$ - $Request_i$;

	Alloc			Request		
	X	Y	Z	X	Y	Z
P0	1	2	1	1	0	3
P1	2	0	1	0	1	2
P2	2	2	1	1	2	0

Deadlock Detection

If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may occur. In this environment, the system must provide:

- An algorithm that examines the state of the system to determine whether a deadlock has occurred.
- An algorithm to recover from the deadlock.

Recovery from Deadlock

When a detection algorithm determines that a deadlock exists, then the system or operator is responsible for handling deadlock problem. There are two options for breaking a deadlock.

1. Process Termination
2. Resource preemption

Process Termination

There are two method to eliminate deadlocks by terminating a process as follows:

1. Abort all deadlocked processes:

2. Abort one process at a time until the deadlock cycle is eliminated

Resource Pre-emption

In resource pre-emption, the operator or system pre-empts some resources from processes and give these resources to other processes until the deadlock cycle is broken.

If pre-emption is required to deal with deadlocks, then three issues need to be addressed:

1. **Selecting a victim:** The system or operator selects which resources and which processes are to be pre-empted based on cost factor.
2. **Rollback:** The system or operator must roll back the process to some safe state and restart it from that state.
3. **Starvation:** The system or operator should ensure that resources will not always be pre-empted from the same process